thws
Technische Hochschule
Würzburg-Schweinfurt

Robotik Praktikum 2

MATLAB Robotics System Toolbox

Kapitel 3 – Roboterbewegung

Sommersemester 2026
Prof. Dr.-Ing. Stefan Friedrich
Based on prior work / slidedeck: Prof. Dr.-Ing. Jean Meyer

Inhalte des 3. Kapitels

- Direkte und inverse Kinematik
- Analytische und numerische Lösungen für die inverse Kinematik
- Trajektorienplanung

  - Trajektorienplanung im Joint Space
  - Trajektorienplanung im Cartesian Space
- Simulation von Roboterbewegungen
- Einbindung von Kollisionskörpern in 3D Modelle

Direkte und inverse Kinematik

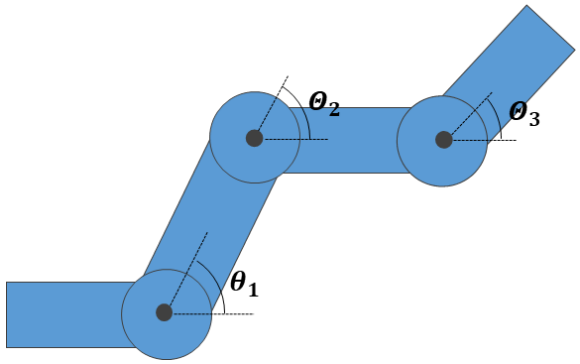
Direkte Kinematik (Vorwärtskinematik) vs. inverse Kinematik:

Direkte Kinematik:

Berechnung der Effektor-Position und Orientierung auf Grundlage von gegebenen Gelenkpositionen

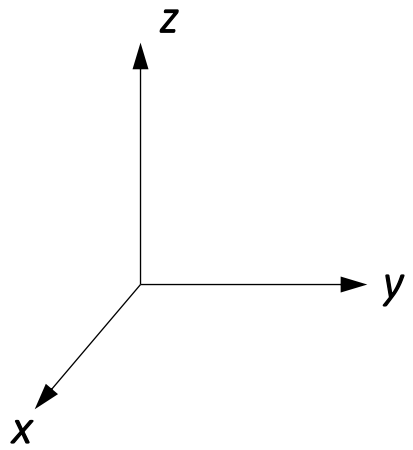
Inverse Kinematik:

Berechnung der Gelenkpositionen (Winkel) auf Grundlage einer gegebenen Effektor-Position und Lage



„Gelenkwinkel Raum“ (Joint Space) $\theta_1, \theta_2, \theta_3, \ldots$

Kartesischer Raum (Cartesian Space) $x, y, z, \ldots$



Direkte und inverse Kinematik

Direkte Kinematik

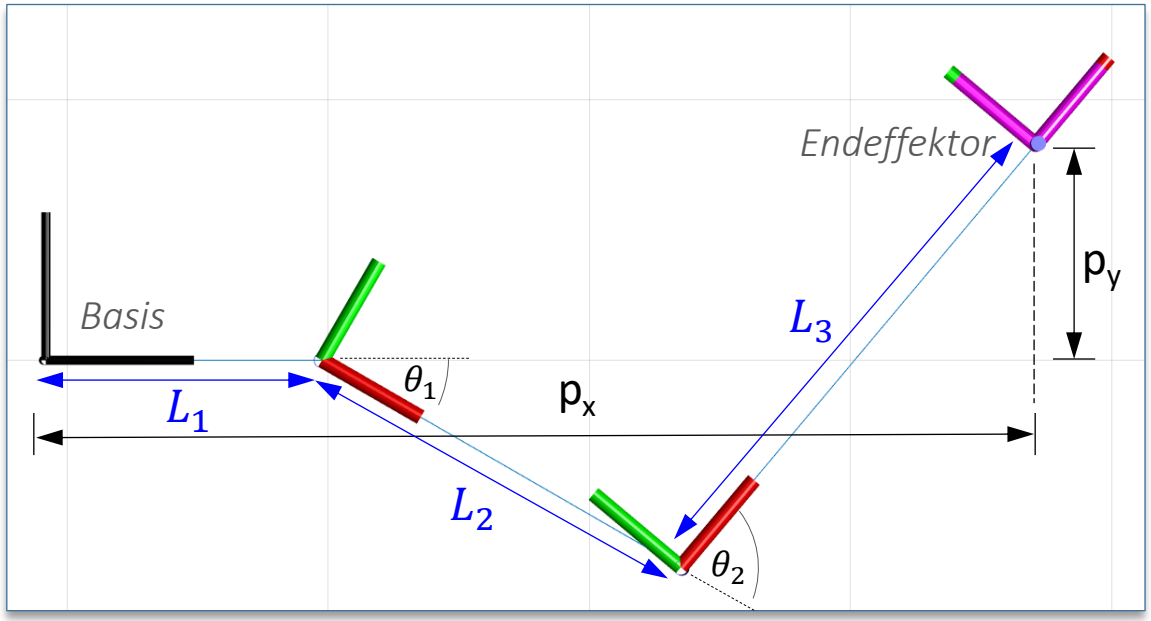
Die kartesischen Koordinaten des Greifers der rechts abgebildeten Kinematik sind eine Funktion der Winkel θ_1 und θ_2 → **Gegeben: $\boldsymbol{\theta_1}$ und $\boldsymbol{\theta_2}$, Gesucht: x und y**

$$x = L_1 + L_2 \cos\theta_1 + L_3 \cos(\theta_1 + \theta_2)$$

$$y = L_2 \sin\theta_1 + L_3 \, sin(\theta_1 + \theta_2)$$

→ Sind die Gelenkwinkel bekannt, kann man sie in obige Gleichung einsetzen und x und y berechnen.

Die Koordinaten x und y können für eine bestimmte Konfiguration auch aus der Transformationsmatix (Endeffektor→Basis) abgelesen werden.



TMatrix = $\boldsymbol{{}^{base}_{endeff.}R}$

0.8660	-0.5000	0	0.7762
0.5000	0.8660	0	0.0072
0	0	1.0000	0
0	0	0	1.0000

$$\boldsymbol{{}^{base}p_{endeff.}} = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix}$$

Direkte und inverse Kinematik

Inverse Kinematik

Die inverse Kinematik beschreibt den umgekehrten Weg zur direkten Kinematik und hat zum Ziel, die Gelenkwinkel zu ermitteln, mit denen eine bestimmte Position im kartesischen Raum erreicht wird

→ **Gegeben: x und y, Gesucht: $\boldsymbol{\theta_1}$ und $\boldsymbol{\theta_2}$**

- Die Bestimmung von θ_1 und θ_2 kann anhand von Gleichungen erfolgen (→ **geschlossene, analytische Lösung**) oder **numerisch**
- Abhängig von der Kinematik kann es sein, dass es keine geschlossene analytische Lösung gibt → Lösung nur numerisch möglich
- Geschlossene Lösungen werden – sofern vorhanden – gegenüber numerischen Lösungen bevorzugt

$$\theta_1 = f(x, y)$$

$$\theta_2 = f(x, y)$$

Direkte und inverse Kinematik

Inverse Kinematik – Geschlossene Lösung

- Entwicklung einer geschlossenen analytischen Lösung in zwei Schritten:

  1. Erzeugung eines inverse Kinematik Objekts mit **`analyticalInverseKinematics`**

  Syntax:
  ```
  analytischeIK = analyticalInverseKinematics(rigidBodyTreeObject);
  ```

  (Es wird automatisch die kinematische Gruppe Basis-Endeffektor gewählt (Basis u. letzter Body im Tree))

  2. Erzeugung einer Funktion zur Beschreibung der inversen Kinematik mit **`generateIKFunction`**

  Syntax:
  ```
  generateIKFunction(analytischeIK,'meineIK');
  ```

- Die Funktion (meineIK) berechnet für eine Pose alle möglichen Gelenkwinkelkonfigurationen

  → Es gibt in der Regel mehrere Konfigurationen, mit denen eine bestimmte Pose erreicht wird

Direkte und inverse Kinematik

Inverse Kinematik – Geschlossene Lösung

- Die Pose muss zur Anwendung in meineIKFunktion als **homogene Transformation** ausgedrückt werden

  → Umwandlung von Vektoren oder Rotationen mit **`XXXXX2tform`**

```
>> EffektorPosition = [0.2 0.2 0.2]; % x,y,z-Position des Endeffektors
>> EffektorPose = trvec2tform(EffektorPosition); % Erzeugung hom. Transf.
>> IKKonfigurationen = meineIK(EffektorPose);
```

	1	2	3	4	5	6
1	1.0015	0.6681	0.9021	-2.1401	1.5704	3.1410
2	1.0015	0.6681	0.9021	1.0015	-1.5704	-5.4453e-04

→ *Konfiguration 1*

→ *Konfiguration 2*

Konfigurationen/Gelenkwinkel A1 bis A6

Direkte und inverse Kinematik

Geschlossene analytische Lösung - Beispiel

Geschlossene Lösung für den Roboter „yaskawaMotomanMH5“ und Berechnung der Konfigurationen für die Position [0.4 m, 0.6 m, 0.4 m].

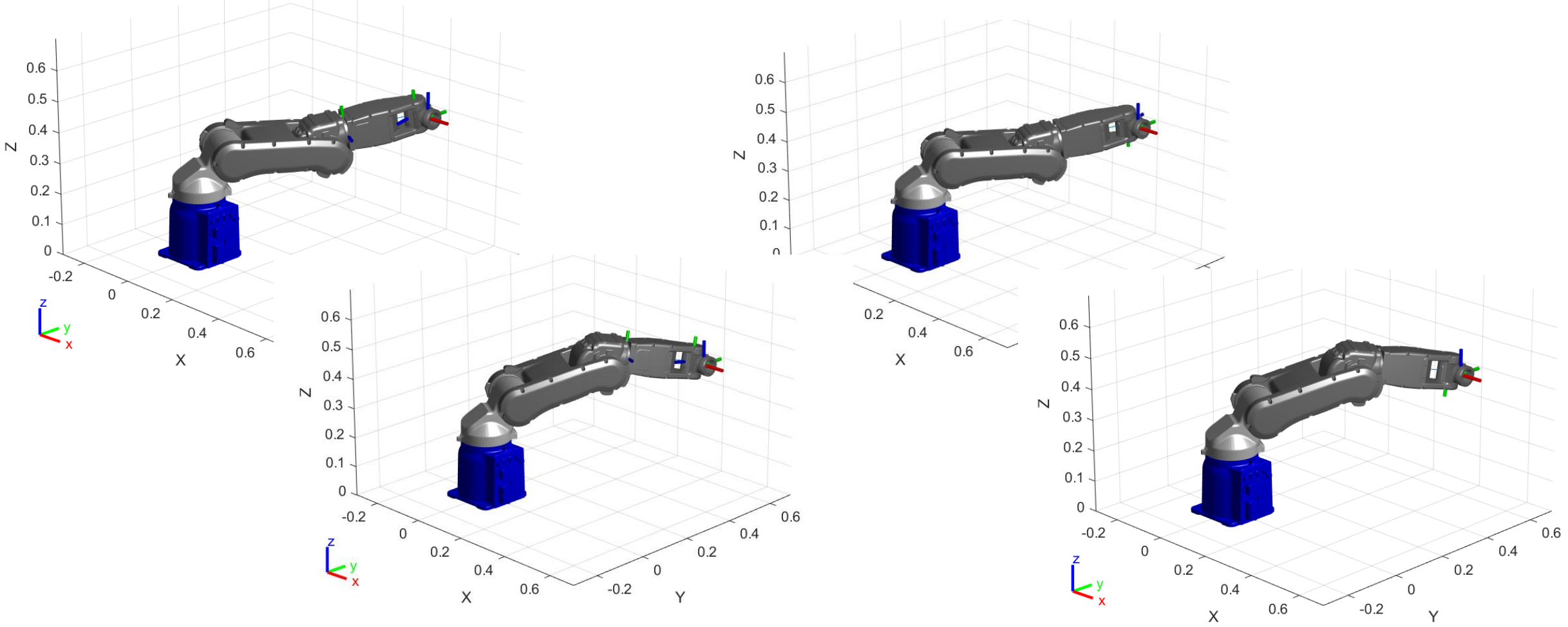
```
>> myRobot = loadrobot('yaskawaMotomanMH5');
>> analytischeIK = analyticalInverseKinematics(myRobot);
>> generateIKFunction(analytischeIK,'meineIK'); % Erzeugung d. geschl. Funkt.
>> EffektorPosition = [0.4 0.6 0.4]; % X = 0.4m, Y = 0.6m, Z = 0.4m
>> EffektorPose = trvec2tform(EffektorPosition); % Umwandlung in hom. Trans
>> IKKonfigurationen = meineIK(EffektorPose);
```

	1	2	3	4	5	6
1	1.0440	1.6107	1.7517	-1.6524	1.0497	-4.5496
2	1.0440	1.3009	1.1297	-1.4720	1.0524	1.3734
3	1.0440	1.6107	1.7517	1.4892	-1.0497	-1.4080
4	1.0440	1.3009	1.1297	1.6696	-1.0524	-1.7682

Es existieren vier Konfigurationen

Direkte und inverse Kinematik

Geschlossene analytische Lösung – Beispiel (Grafische Darstellung der Konfigurationen)



Direkte und inverse Kinematik

Numerische Lösung

- Bei der numerischen Lösung werden die Gelenkwinkel iterativ bestimmt. Das Ergebnis ist eine Näherung.
- Auffinden einer Lösung umfasst i.d.R. viele Iterationsschritte → Höherer Rechenaufwand als bei anal. Lösung
- Eine numerische Lösung liefert eine Lösung, aber nicht alle Lösungen (wie bei analytischer Lösung)
- Bei speziellen/komplexen Kinematiken, für die keine analytische Beschreibung möglich ist, ist eine numerische Lösung die einzige Option
- Erzeugung einer numerischen Lösung (eines Solvers) mit **`inverseKinematics`**

  Syntax:
```
meinIKSolver = inverseKinematics('RigidBodyTree',rigidBodyTreeObj);
```
- Anwendung des Solvers (meineIK) ähnlich zur analytischen Lösung, jedoch mit anderen Parametern. Syntax:

```
[Konfiguration,Info] = meinIKSolver('EndeffektorName',EffPose,weights,initguess);
```

Direkte und inverse Kinematik

Numerische Lösung

```
[Konfiguration,Info] = meinIKSolver('EndeffektorName',EffPose,weights,initguess);
```

- Die **Effektorpose (EffPose)** wird als homogene Transformation beschrieben
- Die **Gewichte (weights, 1x6 Vektor)** geben an, wie stark Abweichungen der Orientierung und Position bei der Berechnung der Pose berücksichtigt werden. Die ersten drei Parameter beziehen sich auf den Orientierungsfehler, die letzten drei auf die Abweichung der Position (x,y,z)
- Der **Startvektor (initGuess)** beschreibt die Anfangskonfiguration, von der aus die Iteration starten soll
- Der Vektor **Konfiguration** enthält die gefundene numerische Lösung
- In der Variable Info enthält nähere Angaben, z.B. die Anzahl der Iterationsschritte

Direkte und inverse Kinematik

Numerische Lösung - Beispiel

Numerische Lösung für den Roboter „yaskawaMotomanMH5“ und Berechnung der Konfigurationen für die Position [0.4 m, 0.6 m, 0.4 m].

```
myRobot = loadrobot('yaskawaMotomanMH5');
meineIK = inverseKinematics('RigidBodyTree',myRobot);
EffektorPosition = [0.4 0.6 0.4]; % X = 0.4m, Y = 0.6m, Z = 0.4m
EffektorPose = trvec2tform(EffektorPosition); % Umwandlung in hom. Trans
weights = [0.5 0.5 0.5 1 1 1];
initguess = myRobot.homeConfiguration;
[Konfiguration,Info] = meineIK('link_t',EffektorPose,weights,initguess);
```

Direkte und inverse Kinematik

Numerische Lösung - Beispiel

Konfiguration

'joint_s'	1.0440
'joint_l'	1.3009
'joint_u'	1.1297
'joint_r'	-1.4720
'joint_b'	1.0524
'joint_t'	1.3734

Es wurde nur eine Konfiguration gefunden

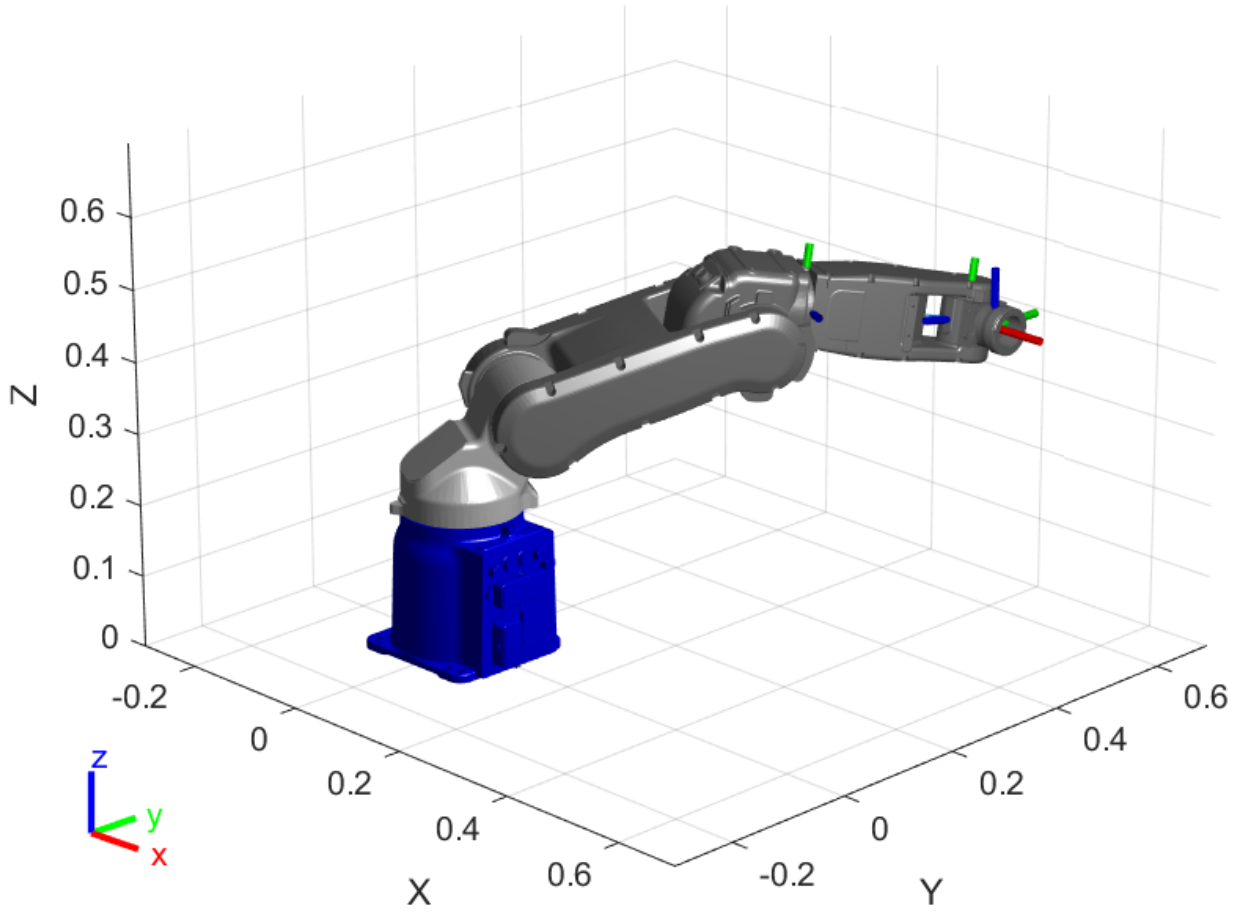
Info

Field	Value
Iterations	31
NumRandomRestarts	0
PoseErrorNorm	1.3591e-09
ExitFlag	1
Status	'success'

Ergebnis nach 31 Iterationen gefunden

1 = Abbruch nach Erreichen des lokalen Minimums

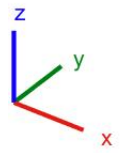
Ergebnis liegt innerhalb der Toleranz

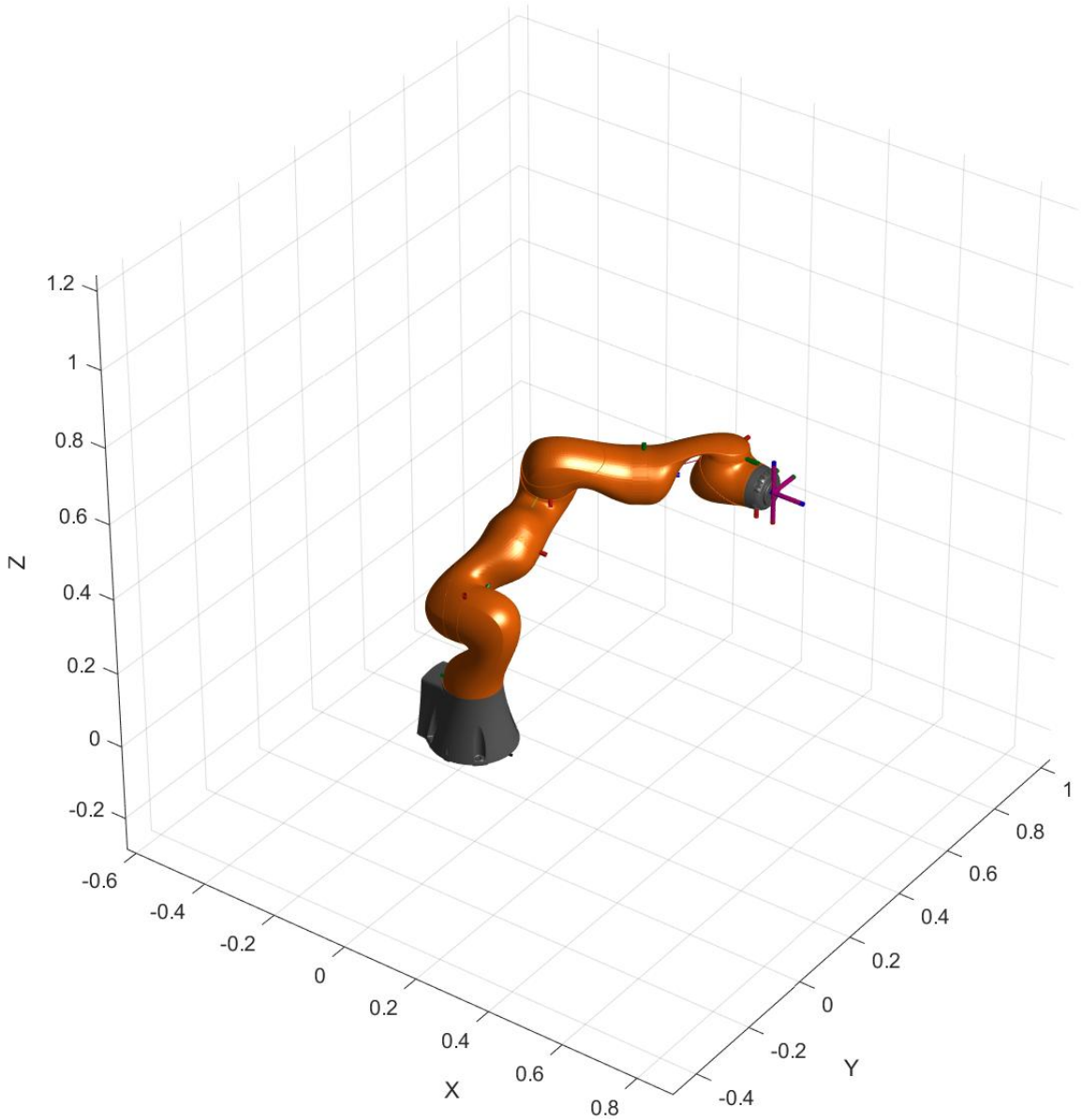


Direkte und inverse Kinematik

Aufgabe 3.1

Bestimmen Sie auf **numerischem Wege** eine Konfiguration für den Roboter „**kukaiiwa7**“, mit der der Endeffektor (Body name: iiwa_link_ee) die **Position [0.4 m, 0.6 m, 0.5 m]** erreicht. Starten Sie die Iteration mit der **Home-Konfiguration** und wählen Sie für Orientierungen und Positionen **gleich große Gewichte**. Stellen Sie den Roboter mit der ermittelten Konfiguration anschließend grafisch dar. Prüfen Sie, ob der Solver regulär (durch Auffinden eines lokalen Minimums) abgebrochen ist (ExitFlag =1?).



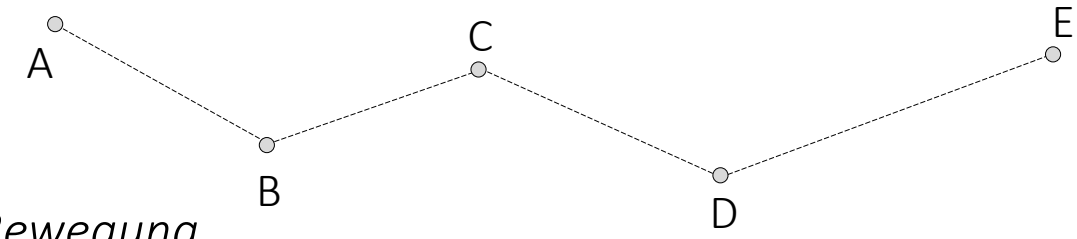


Direkte und inverse Kinematik

Aufgabe 3.1 - Lösung

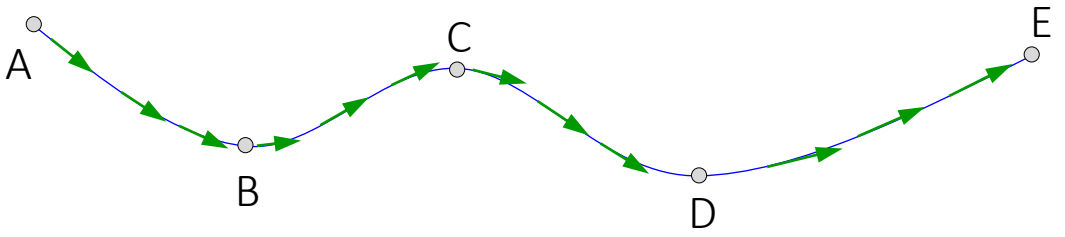
Trajektorienplanung

- Die Trajektorienplanung baut auf der Pfadplanung auf
- **Pfadplanung:** Abfolge von Punkten, über die der Roboter (kollisionsfrei) vom Start- zum Endpunkt gelangt



Kriterium: Kollisionsfreie Bewegung

- **Trajektorienplanung:** Interpolation zwischen Pfadpunkten und gleichzeitige Bestimmung der Geschwindigkeit und der Beschleunigung auf jedem Abschnitt



Übliches Kriterium: Minimierung der Zeit zwischen den Punkten

Trajektorienplanung

- Die Trajektorienplanung kann im **Gelenkwinkelraum** oder im **Kartesischen Raum** erfolgen
  - Trajektorienplanung im **Gelenkwinkelraum**: Interpolation der Gelenkwinkel
  - Trajektorienplanung im **Kartesischen Raum**: Interpolation der Kartesischen Koordinaten
- Abhängig von der Planung im Gelenkwinkel- oder Kartesischen Raum unterscheiden sich die Planungsschritte:
  - Planung im Gelenkwinkelraum:

Ggf. Bestimmung der Gelenkwinkel aus kartesischen Koordinaten der Punkte (Inverse Kinematik) → *Interpolation zwischen Gelenkwinkeln der Punkte*

  - Planung im kartesischen Raum:

Interpolation zwischen kartesischen Koordinaten der Punkte → *Bestimmung der Gelenkwinkel aus interpolierten kartesischen Punkten (Inverse Kinematik)*

Funktionen zur Trajektoriengenerierung

Trapezförmige Trajektorie

- Generierung einer Trajektorie mit **trapezförmigem Geschwindigkeitsprofil** mit **`trapveltraj`**

  Syntax: `[q,qd,qdd,tSamples,pp] = trapveltraj(wayPoints,numSamples)`

  Anzahl der ***interpolierten Punkte***

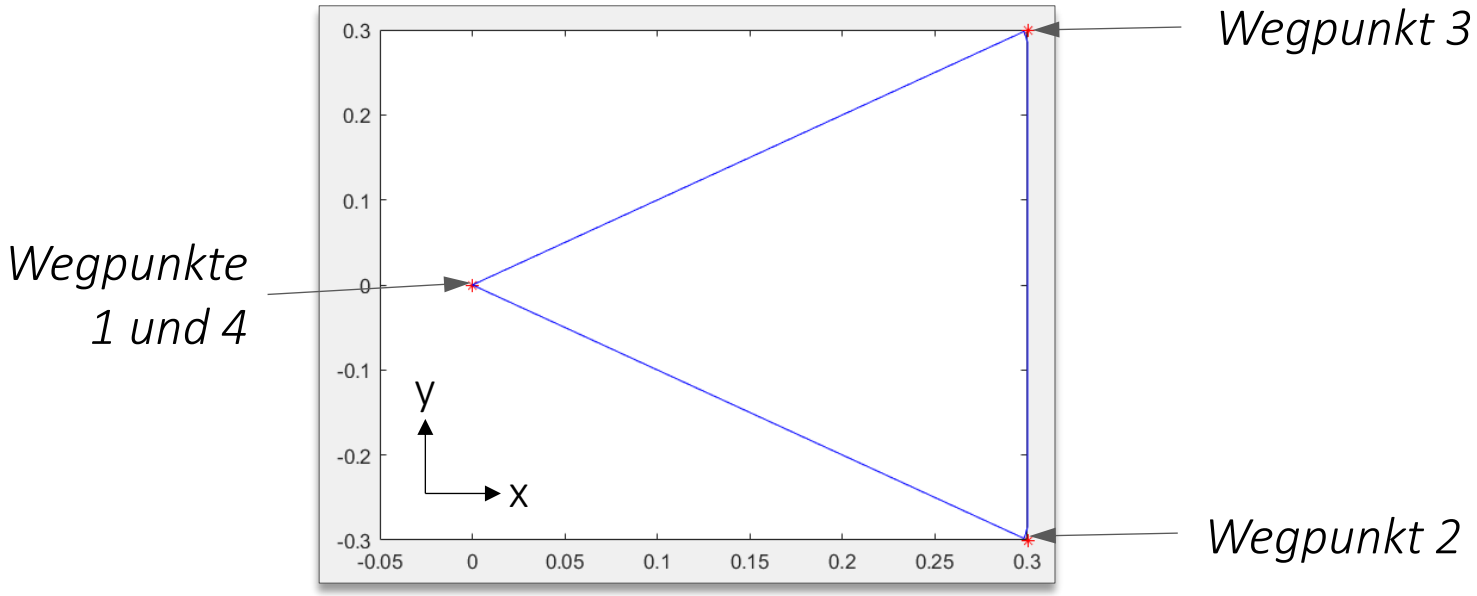
  Wegpunkte*, entweder als Gelenkwinkelpositionen oder kartesische Positionen*

- Bedeutung der Trajektorienparameter:
  - `q`: Positionen der Trajektorie
  - `qd`: Geschwindigkeit (1. Ableitung von q)
  - `qdd`: Beschleunigung (2. Ableitung von q)
  - `tSamples`: Zeitmarken der einzelnem Positionen
  - `pp`: Polynomkoeffizienten

Funktionen zur Trajektoriengenerierung

Trapezförmige Trajektorie (*Beispiel: Trajektoriengenerierung im Kartesischen Raum*)

```
>> Wegpunkte = [0 0.3 0.3 0; 0 -0.3 0.3 0]; % xy-Wegpunkte
>> [q,qd,qdd,tSamples,pp] = trapveltraj(Wegpunkte,100); % 100 Interp. Punkte
>> plot(Wegpunkte(1,:),Wegpunkte(2,:),'*r',q(1,:),q(2,:),'b');
```

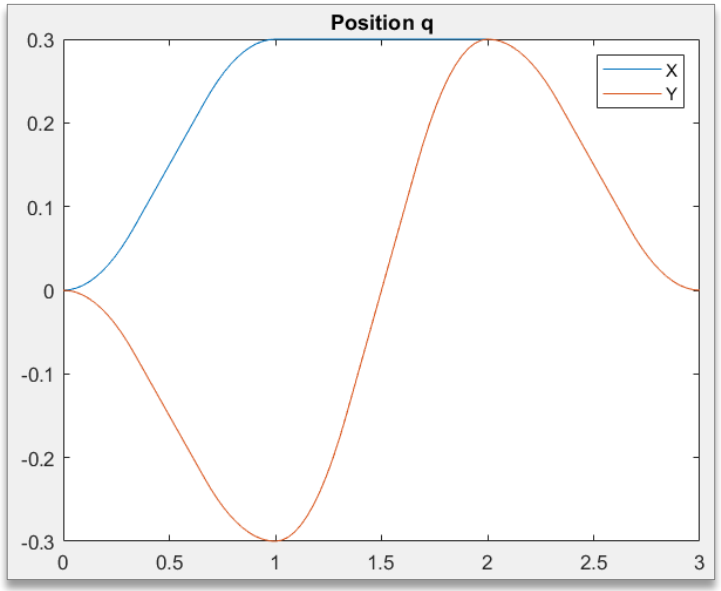


Funktionen zur Trajektoriengenerierung

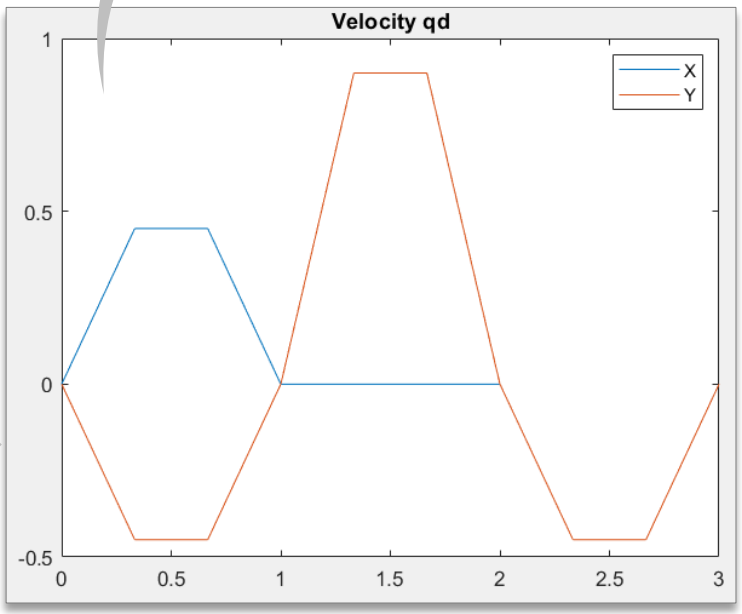
Trapezförmige Trajektorie (*Beispiel - Fortsetzung*)

```
>> % Plot der Position
>> figure(2)
>> plot(tSamples,q);
>> title('Position q');
>> legend('X','Y');

>> % Plot d Geschwindigkeit
>> figure(3)
>> plot(tSamples,qd);
>> title('Velocity qd');
>> legend('X','Y');
```



Trapezförmiges Geschwindigkeitsprofil



Funktionen zur Trajektoriengenerierung

Aufgabe 3.2

Für einen Industrieroboter mit 4 Drehgelenken (A1 .. A4) soll eine Trajektorie zwischen zwei Wegpunkten im Gelenkwinkelraum generiert werden. Die Startposition lautet (in rad) [0; pi/4; -pi/2; 0] und die Zielposition lautet [-pi/4; -pi/4; pi/2; pi/6].

Erzeugen Sie eine Trajektorie mit trapezförmigem Geschwindigkeitsprofil mit 200 Abschnitten bzw. Samples. Stellen Sie die Gelenkwinkel sowie die Geschwindigkeit als Funktion der Zeit in zwei getrennten Plots dar.

Hinweis zur Syntax:

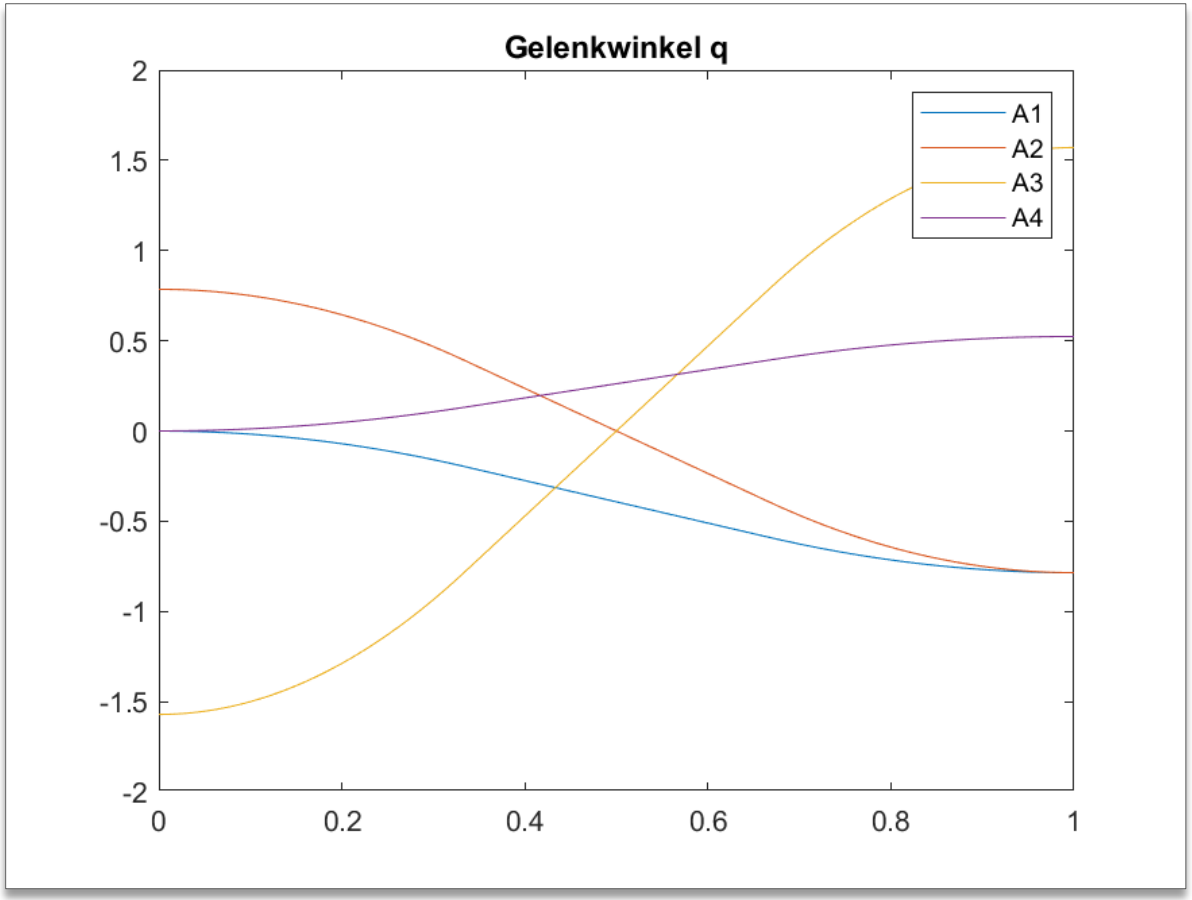
```
[q,qd,qdd,tSamples,pp] = trapveltraj(wayPoints,numSamples)
```

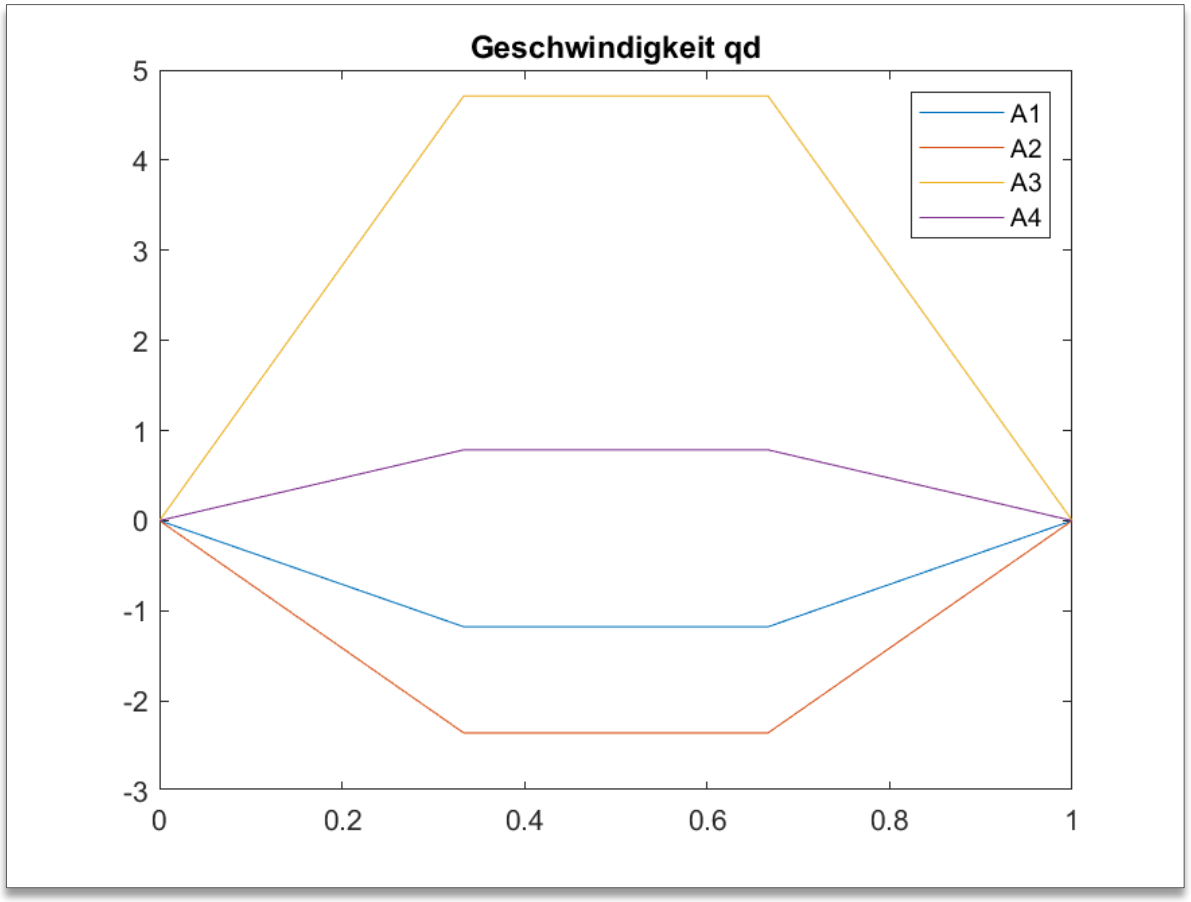
Funktionen zur Trajektoriengenerierung

Aufgabe 3.2 – Lösung

Funktionen zur Trajektoriengenerierung

Aufgabe 3.2 – Lösung





Funktionen zur Trajektoriengenerierung

Trapezförmige Trajektorie

- Für die Berechnung der Trajektorie mit der Funktion **`trapveltraj`** können weitere Randbedingungen formuliert werden, z.B.:
  - Maximalgeschwindigkeit → `PeakVelocity`
  - Beschleunigung → `Acceleration`
  - Zeitliche Dauer je Segment → `EndTime`
  - Dauer der Beschleunigungsphase → `AccelTime`

*Aus mathematischen Gründen können **maximal zwei Randbedingungen** gleichzeitig gesetzt/erfüllt werden.*

- Bei dynamischen Anwendungen macht es Sinn, die **Maximalgeschwindigkeit** und die **Beschleunigung** als Randbedingungen zu setzen, entsprechend den Leistungskennwerten im Datenblatt des Roboters
- *Beispiel*:

2 rad/s 8 rad/s²

```
[q,qd,qdd,tSamp,pp] = trapveltraj(Wegpunkte,100,'PeakVelocity',2,'Acceleration',8);
```

Trajektoriengenerierung

- Neben der Trajektorien mit trapezförmigem Geschwindigkeitsprofil kommen weitere Funktionen zur Trajektoriengenerierung zum Einsatz, z.B.:
  - B-Splines → **`bsplinepolytraj`**
  - Kubische Polynome (Polynome 3. Ordnung) → **`cubicpolytraj`**
  - Polynome 5. Ordnung → **`quinticpolytraj`**
- Beispiel: **Kubische Polynome**
  - Ansatz: Der Verlauf eines Gelenkwinkels zwischen zwei Wegpunkten ist ein Polynom 3. Ordnung

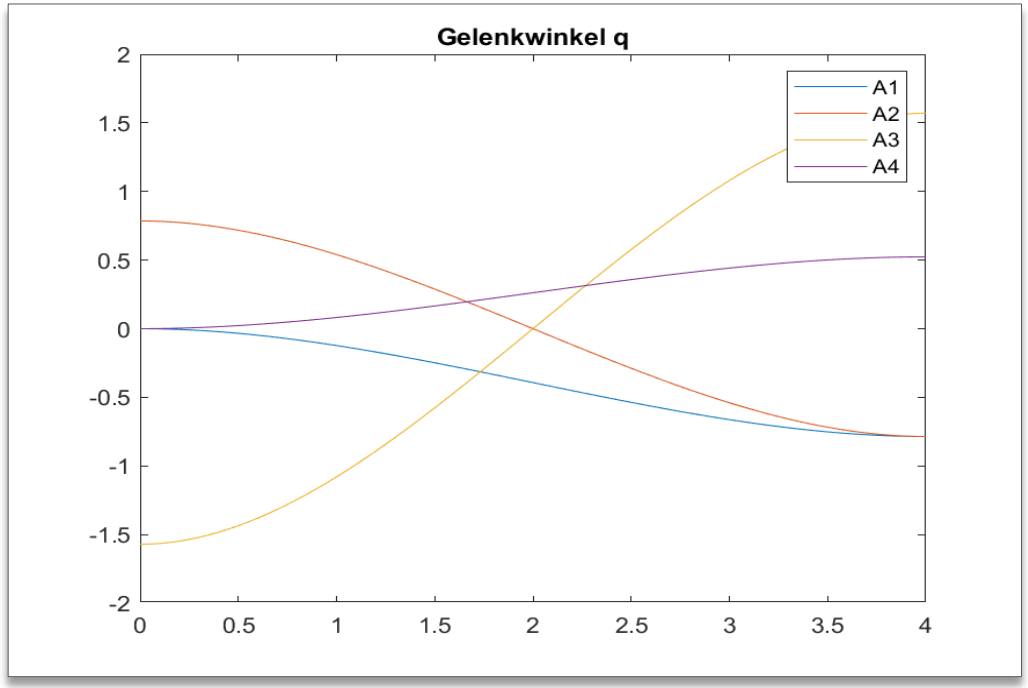
$$\theta(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 \longrightarrow \text{Kubisches Polynom (Polynom 3. Grades)}$$

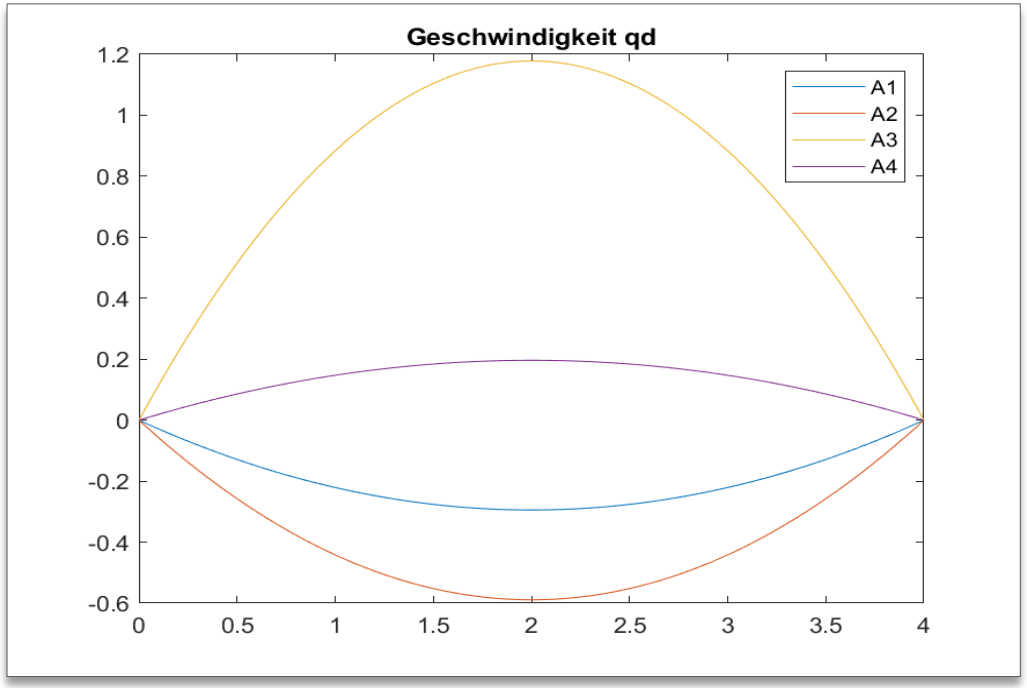
  - Vier unbekannte Parameter (a_1,a_2,a_3,a_4), die aus Randbedingungen ermittelt werden
  - Randbedingungen: Definierte Wegpunkte θ_0 und θ_1 und Geschwindigkeiten $\dot{\theta}_0 = 0$, $\dot{\theta}_1 = 0$

Trajektoriengenerierung

Kubische Polynome

- Syntax: `[q,qd,qdd,pp] = cubicpolytraj(Wegpunkte,Zeitpunkte,SampleZeiten);`
- Kubische Trajektorie für Wegpunkte aus Aufgabe 3.2:

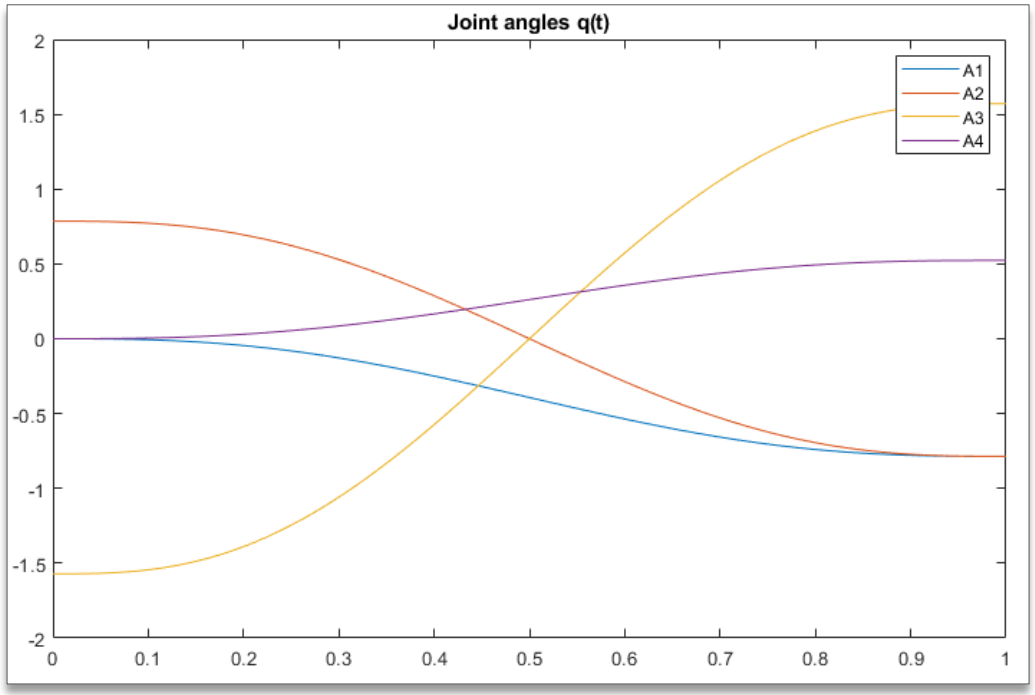


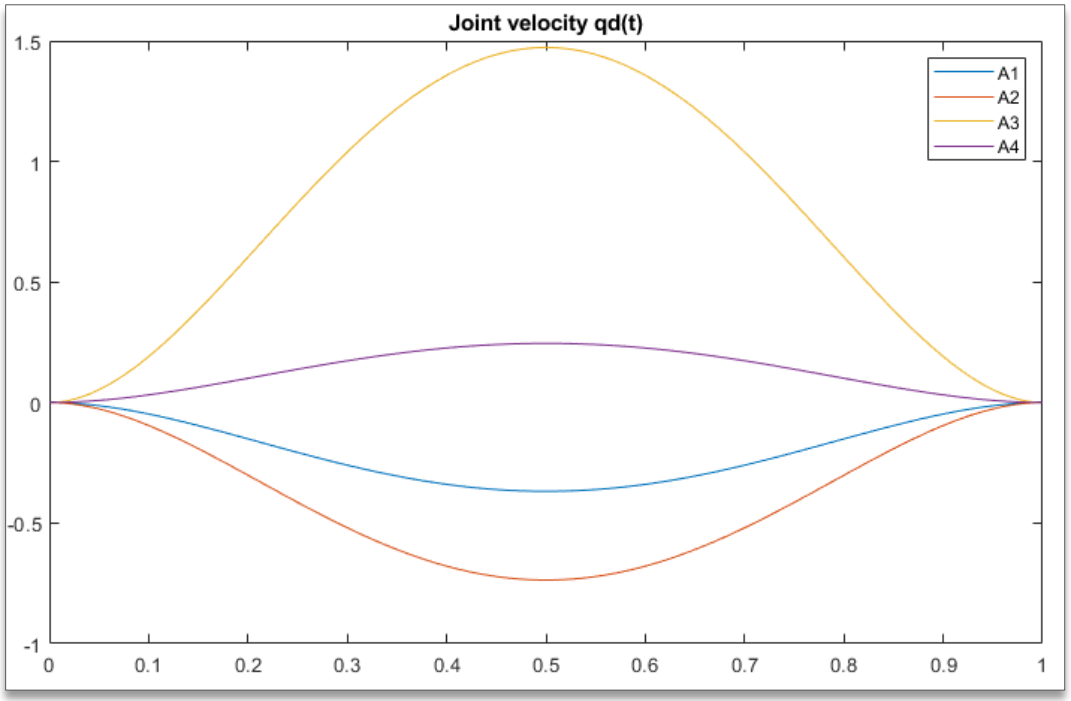


Trajektoriengenerierung

Quintische Polynome

- Syntax: `[q,qd,qdd,pp] = quinticpolytraj(Wegpunkte,Zeitpunkte,SampleZeiten);`
- Quintische Trajektorie für Wegpunkte aus Aufgabe 3.2:





Simulation von Roboterbewegungen

- Um Roboterbewegungen realistisch zu simulieren, muss neben den interpolierten Bahnpunkten auch die Geschwindigkeit berücksichtigt werden → Ergebnisse aus der Trajektorienplanung
- Zur Simulation bietet sich der interaktive Darstellungsmodus an (→ interactiveRigidBodyTree)
  → Einfache (interaktive) Änderung und Speicherung von Konfigurationen
  → Konfigurationen können über Variablen im Workspace direkt geändert und sichtbar gemacht werden
- Vorgehen zur Simulation:
  1. Roboter laden und Darstellung mit `interactiveRigidBodyTree`
  2. Speicherung von Wegpunkten/Konfigurationen
  3. Erzeugung einer Trajektorie, z.B. mit `trapveltraj`
  4. Simulation der Roboterbewegung in der interaktiven Darstellung

Simulation von Roboterbewegungen

Beispiel – Simulation

1. Roboter laden und Darstellung mit `interactiveRigidBodyTree`

```
>> myRobot = loadrobot(„kukaIiwa14“);
>> iaDarstellung = interactiveRigidBodyTree(myRobot);
```

2. Speicherung von Wegpunkten/Konfigurationen

```
>> addConfiguration(iaDarstellung);
```

3. Erzeugung einer Trajektorie

```
>> AnzahlKonfigurationen = size(iaDarstellung.StoredConfigurations,2);
>> AnzahlSamples = 50*(AnzahlKonfigurationen-1);
>> [q,qd,qdd,tSamp] = …
   trapveltraj(iaDarstellung.StoredConfigurations,AnzahlSamples);
```

Simulation von Roboterbewegungen

Beispiel – Simulation

4. Simulation der Roboterbewegung in der interaktiven Darstellung

Sample 2, weil Sample 1 sofort gezeigt wird. Erst danach kann eine zeitlich kontrollierte Bildwiedergabe erfolgen.

```
>> rateController = rateControl(AnzahlSamples/(max(tSamp) - tSamp(2)));
>> for i = 1:AnzahlSamples
       iaDarstellung.Configuration = q(:,i);
       waitfor(rateController);
   end
```

Hinweis: rateControl dient dazu, die Bilder kontrolliert im Einklang mit dem realen Zeitverlauf anzuzeigen.

Syntax: `rateObject = rateControl(Rate)`

Frequenz in Hz

Simulation von Roboterbewegungen

Aufgabe 3.3

Laden Sie den Roboter „kukaliwa14“. Lassen Sie den Roboter mit `interactiveRigidBodyTree` anzeigen. Bewegen Sie den Roboter mittels „click and drag“ und speichern Sie vier verschiedene Konfigurationen (mit `addConfiguration`). Erzeugen Sie anschließend eine Trajektorie, die alle vier Punkte nacheinander verbindet (mit 100 Samples je Abschnitt). Limitieren Sie die PeakVelocity auf 2 rad/s. Simulieren Sie anschließend die Roboterbewegung.

Hinweis: Schleife zur Simulation

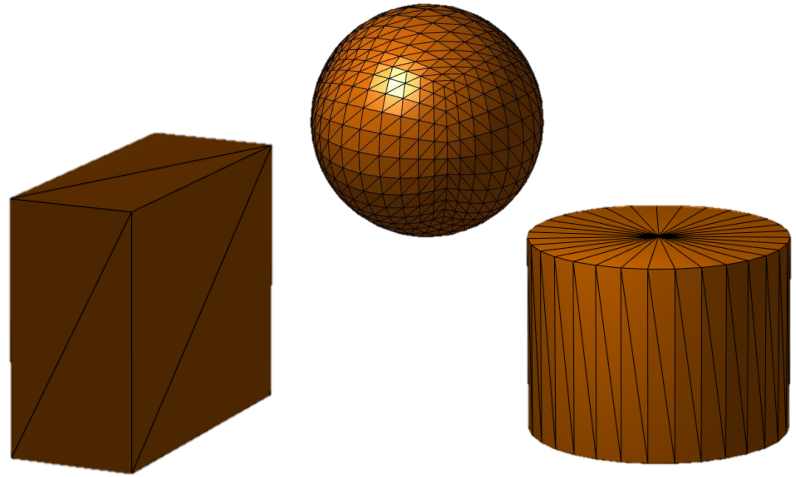
```
rateController = rateControl(AnzahlSamples/(max(tSamp) - tSamp(2)));
for i = 1:AnzahlSamples
    iaDarstellung.Configuration = q(:,i);
    waitfor(rateController);
end
```

Simulation von Roboterbewegungen

Aufgabe 3.3 – Lösung

Kollisionskörper

- Roboter arbeiten nie im leeren Raum, sondern in Umgebungen mit verschiedenen Objekten
- Einbindung von Objekten in 3D Modelle als „Kollisionsgeometrien“
- Verfügbare Geometrien:
  - Quader → **`collisionBox(Länge X,Breite Y,Höhe Z)`**
  - Zylinder → **`collisionCylinder(Radius,Höhe)`**
  - Kugel → **`collsionSphere(Radius)`**



- *Beispiel*:

```
>> myBox = collsionBox(0.5,0.3,0.1); % Kollisionsgeometrie erzeugen
>> myBox.Pose = trvec2tform([0 0 -0.5]); % Verschiebung um 0.5 m in -Z
>> [Achsen,patchObjBox] = show(myBox); % Anzeigen der Kollisionsgeometrie
```

PatchObjekt über das Änderungen der Darstellung (z.B. Farbe) vorgenommen werden können

Koordinatenachsen, in denen die Kollisionsgeometrie dargestellt wird

Kollisionskörper

- Die Geometrien werden im **Ursprung** (X=0,Y=0,Z=0) erzeugt. Veränderung der Pose über **`.Pose`** (→ homogene Transformation). *Beispiel*:

```
>> myBox.Pose = trvec2tform([0 0 -0.5]);
```

- Die Kollisionsgeometrien an sich sind farblos. Änderungen der Darstellung können nur am **PatchObjekt** der Kollisiongeometrie vorgenommen werden

Koordinatenachsen der Darstellung (→ hier nicht benötigt)

```
>> [~,patchObjBox] = show(myBox);
>> patchObjBox.FaceColor = [0 0 1];
>> patchObjBox.EdgeColor = [0 0 0];
```

→ *Änderung der (Flächen-)Farbe des Quaders*

→ *Änderung der Kantenfarbe. Keine Farbe =* `'none'`

RGB Farbschema [Rot Grün Blau]

- Darstellung der Kollisionsgeometrie in bestimmten Koordinatenachsen:

```
>> [~,patchObjBox] = show(myBox,'Parent',ax);
```

→ *Sinnvoll bei Darstellung mehrerer Objekte*

Kollisionskörper

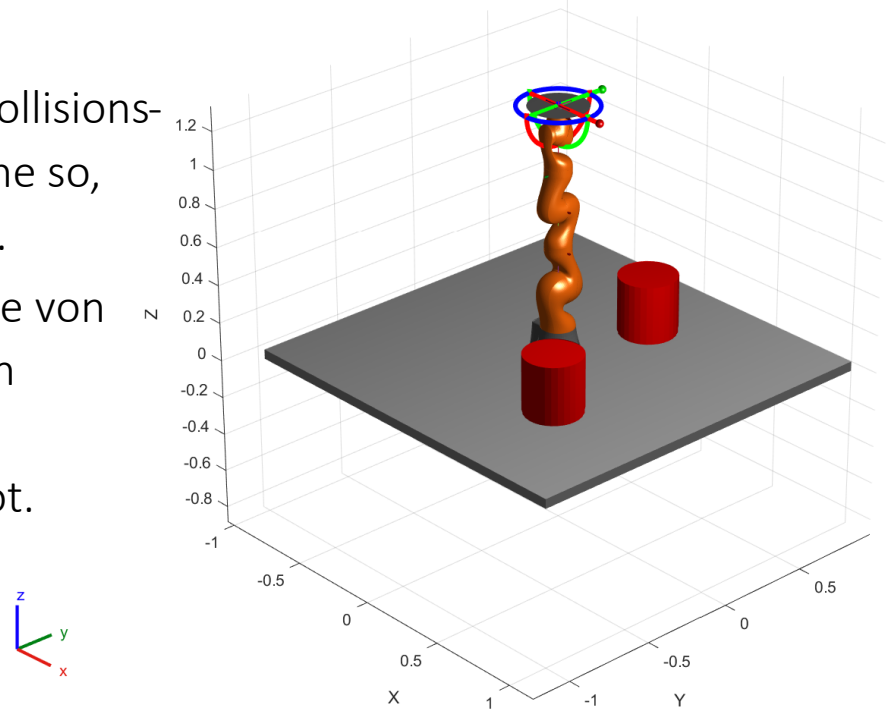
Aufgabe 3.4

Erzeugen Sie ein Script mit dem Namen „Roboterumgebung.m“, in das Sie alle nachfolgenden Schritte aufnehmen. Importieren/laden Sie die das Roboter-Modell `"kukaiiwa14"` (mit `loadrobot`) und nennen Sie es „myRobot“. Stellen Sie den Roboter anschließend mit `interactiveRigidBodyTree` dar und nennen Sie das (interaktive) Objekt „myInteractiveRobot“. Speichern Sie das Skript und rufen Sie es auf, um sich zu vergewissern, dass sich der „Fuß“ des Roboters bei den Koordinaten (0, 0, 0) befindet. Diese Koordinaten markieren den späteren Ursprung des „Base-Koordinatensystems“ des Roboters.

Stellen Sie den Roboter auf eine Arbeitsfläche, indem Sie eine quaderförmige Kollisions-geometrie mit den Abmaßen LxBxH = 2x2x0.05. Verschieben Sie die Arbeitsfläche so, dass die Oberseite bei Z=0 liegt und mit der Unterseite des Roboters abschließt.

Erzeugen Sie zwei Zylinder mit einem Durchmesser von je 30 cm und einer Höhe von 40 cm. Positionieren Sie die Zylinder auf der Tischfläche bei den x,y-Koordinaten (0.1m, 0.5m) und (0.4m, -0.4m).
Ändern Sie die Farbe der Tischplatte in Hellgrau und die Farbe der Zylinder in rot.

Rufen Sie das Skript auf und überprüfen Sie das Ergebnis.



Kollisionskörper

Aufgabe 3.4 – Lösung

Kollisionsprüfung

- Es gibt zwei Arten von Kollisionen: Roboter Selbstkollision; Kollision mit anderen Objekten in der Umgebung des Roboters
- Man kann eine spezifische Pose eines Roboters auf Kollisionen (ggfs. mit anderen Objekten) prüfen wie folgt:

```
[isColliding, separationDist, witnessPts] = ...
        checkCollision(robot, config, worldObjects, Name, Value)
```

- Input:
  - Erster Parameter **`robot`** ist ein Objekt vom Typ rigid-body tree
  - Der zweite Parameter **`config`** ist die zu prüfende Konfiguration des Roboters (= Vektor mit Gelenkspositionen)
  - Der dritte Parameter **`worldObjects`** ist ein cell array, das die Kollisionsgeometrien wie z.B. collisionBoxes, collisionSpheres, etc. beinhaltet.

    Achtung: Die Eigenschaft **`Pose`** jedes dieser Weltobjekte bezieht sich relative zur "base" des **`robot`** Modells
  - Optionale "Name"/"Value" Paare, um die Suchtiefe und –eigenschaften einzustellen; siehe Dokumentation.

    Oftmals ist es nützlich, hier noch **`"IgnoreSelfCollision", "on", "Exhaustive","on"`** anzugeben.
- Outputs:
  - **`isColliding`**: Vektor mit boolschen Werten zum Ergebnis; Eintrag 1 für Eigen- und Eintrag 2 für Fremdkollision
  - **`separationDist`**, **`witnessPts`**: Matrizen, die genauere Informationen zu den Abständen zwischen den Objekten beinhalten sowie die zugehörigen "witness Points" (beweisende Punkte), also die nähesten Punkte in [x,y,z] bezogen auf die Oberflächen aller geprüften Kollisionskörper. Im Falle von Kollision sind die zugehörigen Einträge **`NaN`**. Details in der Dokumentation.

Kollisionsprüfung – Beispiel: Vorbereitung der Szene

`[isColliding,separationDist,witnessPts] = . . .`

`checkCollision(robot,config,worldObjects, Name, Value)`

Beispiel:

```
>> myRobot = loadrobot("kukaiiwa14");
>> myRobot.DataFormat = "column";
>> rng(42); myRobotConfig = myRobot.randomConfiguration;

>> % Objekte in der Umgebung erstellen
>> myBox = collisionBox(0.2,0.6,0.4);
>> myBox.Pose = trvec2tform([0 -0.5 0]);
>> myCylinder = collisionCylinder(0.1,0.5);
>> myCylinder.Pose = trvec2tform([-0.4 0.4 -0.1]);

% Visuelle Prüfung
>> figure; show(myRobot, myRobotConfig); hold on; show(myBox); show(myCylinder);
```

Kollisionsprüfung – Beispiel: Prüfung inkl. möglicher Eigenkollision

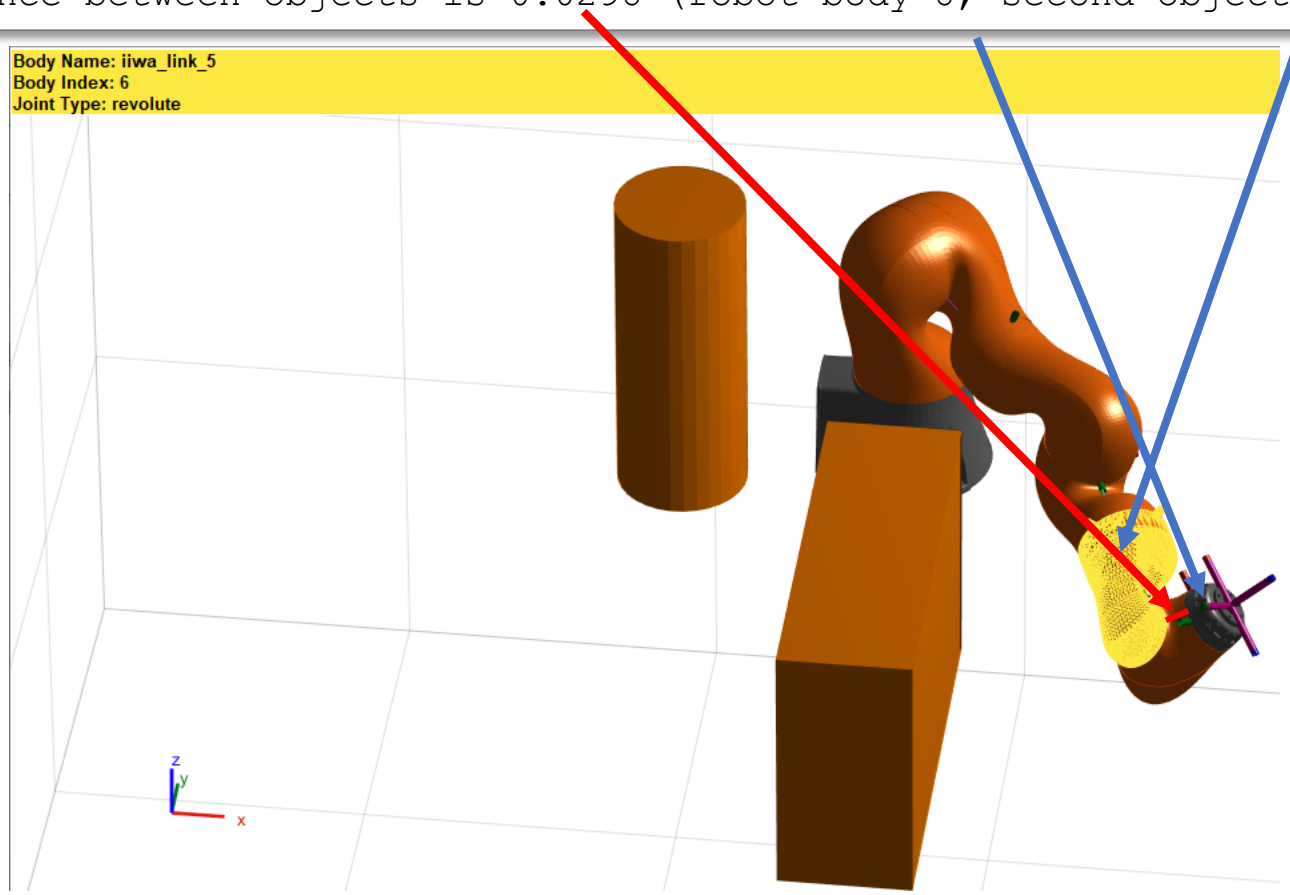
```
>> % Berechnen, ob die gegebene Konfiguration zur Kollision mit einem der Objekte oder sich selbst führt
>> [collides, sepDist, ~] = ...
>>     checkCollision(myRobot, myRobotConfig, { myCylinder, myBox}, ...
>>                    "IgnoreSelfCollision", "off", ...
>>                    "SkippedSelfCollisions", "adjacent", ...
>>                    "Exhaustive", "on");
>> % Für Kollisionsergebnis mit den Weltobjekten braucht man den zweiten Eintrag
>> if collides(2)
>>     disp("Robot collides with environment.");
>> end
>> % Alternativ kann man prüfen, ob irgendwo in der separation Distance Matrix ein NaN Eintrag ist
>> if any(any(isnan(sepDist)))
>>     disp('Found a NaN so robot collides.');
>> else
>>     disp('Robot does not collide.');
>>     % Diejenigen Kollisionskörper mit dem kleinsten Abstand heraussuchen und Ergebnis ausgeben
>>     [indRobotBody, indSecondBody]=find(sepDist==min(min(sepDist)),1);
>>     fprintf(['Minimum distance between objects is ' ...
>>              '%.4f (robot body %d, second object %d).\n'], ...
>>             min(min(sepDist)), indRobotBody, indSecondBody);
>> end
```

Kollisionsprüfung – Beispiel: Prüfung ohne Eigenkollision

```
>> % Berechnen, ob die gegebene Konfiguration zur Kollision mit einem der Objekte führt
>> [collides, sepDist, ~] = ...
>>     checkCollision(myRobot, myRobotConfig, { myCylinder, myBox}, ...
>>                    "IgnoreSelfCollision", "on", ...
>>                    "Exhaustive", "on");
>> % Hier ist das Ergebnis ein Skalar, da sowieso nur die Umgebung geprüft wird
>> if collides
>>     disp("Robot collides with environment.");
>> end
>> % Auch hier können wir die NaNs suchen (dann liegt eine Kollision vor)
>> if any(any(isnan(sepDist)))
>>     disp('Found a NaN so robot collides.');
>> else
>>     disp('Robot does not collide.');
>>     % Heraussuchen, welche zwei Körper zwischen Roboter und Umgebung sich am nähesten kommen
>>     [indRobotBody, worldBody]=find(sepDist==min(min(sepDist)),1);
>>     fprintf(['Minimum distance between robot and world objects is ' ...
>>              '%.4f (robot body %d, second object %d).\n'], ...
>>              min(min(sepDist)), indRobotBody, worldBody);
>> end
```

Kollisionsprüfung – Beispiel: Ergebnis (inkl. Eigenkollision)

```
Robot does not collide.
Minimum distance between objects is 0.0298 (robot body 8, second object 6).
```



Kollisionsprüfung – Beispiel: Ergebnis (exkl. Eigenkollision)

```
Robot does not collide.
Minimum distance between robot and world objects is 0.0506 (robot body 4, second object 2).
```

